

Week 2 cheat sheet: functions, arrays, figures, calculus

Quantitative Methods in Neuroscience, 2026-27. Keep it next to you while you work.

Writing a function

```
def sigmoid(x, slope=1.0, midpoint=0.0):
    """One line saying what it does."""
    return 1 / (1 + np.exp(-slope * (x - midpoint)))

sigmoid(0.5)          # slope and midpoint
default
sigmoid(0.5, slope=4)  # or say which you mean
```

Your own module

```
from src.qmn_utils import sigmoid
```

Helpers live in a .py file so that two notebooks cannot end up with two versions of the same function.

Making arrays

```
np.array([1.0, 2.0, 3.0])
np.linspace(0, 6, 600)  # 600 points, ends included
np.arange(0, 6, 0.01)   # step of 0.01, end excluded
x.shape, x.size
```

Arithmetic without loops

```
y = 2 * x + 1          # every element at once
np.exp(x), np.log(x), np.sqrt(x)
np.sin(x), np.cos(x), np.pi, np.e
np.abs(x), x.max(), x.mean()
```

Picking parts out

```
x[0], x[-1], x[10:20]
mask = x > 2           # an array of True and False
x[mask]                # only where it is True
np.argmax(y)           # position of the largest value
```

The functions of the week

exponential decay	$y = A * \exp(-t / \tau)$
Gaussian	$y = \exp(-(x - \mu)^2 / (2 * \sigma^2))$
sigmoid	$y = 1 / (1 + \exp(-k * (x - x_0)))$
sine	$y = A * \sin(2 * \pi * f * t)$

Anatomy of a figure

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y, lw=1.5, label="signal")
ax.set_xlabel("time (s)")
ax.set_ylabel("amplitude")
ax.set_title("what this panel shows")
ax.legend()
fig.tight_layout()
plt.show()
```

Other kinds of panel

<code>ax.scatter(x, y)</code>	one dot per observation
<code>ax.hist(v, bins=30)</code>	a distribution
<code>ax.bar(names, heights)</code>	a value per category
<code>ax.errorbar(x, y, yerr=e)</code>	a value with its uncertainty
<code>ax.axhline(0), ax.axvline(0)</code>	a reference line

Several panels, and saving

```
fig, axes = plt.subplots(2, 1, figsize=(8, 5),
                          sharex=True)
axes[0].plot(t, signal)
axes[1].plot(t, rate)
fig.savefig("figure.png", dpi=200,
           bbox_inches="tight")
```

Derivatives, numerically

```
dy = np.gradient(y, x)  # slope at every point
d2y = np.gradient(dy, x) # curvature
```

Positive slope rising, zero at a peak or a trough, and the second derivative says which of the two.

Integrals, numerically

```
area = np.trapezoid(y, x)  # the total
running = np.cumsum(y) * dx # the accumulation
```

On older NumPy the first one is called `np.trapz`.

Measuring a decay

```
level = baseline + (peak - baseline) / np.e
# tau is the time from the peak to that level
```

The **1/e rule**: after one time constant, a decay has fallen to about 37% of the way from baseline to peak.